DAA code complexities
Line-by-line explanation and calculations

Based on the DAA (Group A) code in the attached LP3_Complete_Practical.pdf. Covers A1-A6 and A7a, A7b, A7c only.

What this PDF contains

For every algorithm: the code statement, how often it runs, the cost of that statement, the total-time calculation, a numerical example, and the memory calculation. Driver and benchmarking code are explained separately so their costs are not confused with the algorithm.

How to read the calculations

Time = sum of (execution frequency × cost per execution). Add consecutive blocks; multiply genuinely nested loop counts. For recursion, count all calls for time but only simultaneously active calls for stack space. Keep the dominant growing term.

O(f) is an upper bound; Θ(f) is a tight growth rate. “Amortized O(1)” means append is constant on average across many appends; a single resize may cost more. Auxiliary space excludes supplied inputs. Output storage is identified explicitly.

Counting model used throughout

Use the standard DAA unit-cost model: indexing, len, comparisons, and arithmetic on bounded-size numbers cost O(1). Python integers can grow: Fibonacci and binomial values have increasing bit lengths, so their exact big-integer running times can be higher. These are operation-count analyses, not exact CPU instruction counts or measured seconds.

Loop tables count body visits. A loop with k visits also has an exhaustion check; this changes constants, not the asymptotic result. Definitions, comments and imports add no input-dependent algorithmic work after setup. Line numbers below are local to each displayed function, not the original PDF.

Scope of variable inputs

The demonstrations contain fixed sample data, $n \leq 30$ and an 8×8 board. Complexity describes the generalized algorithms as their input sizes grow. Taken literally with all demo limits fixed, those bounded runs are constant-sized.

A1 | Iterative Fibonacci

Line	Code statement	Frequency and cost / explanation
1	`def iterative(n):`	Defines the function; analyze one call with n ≥ 0.
2	`a, b = 0, 1`	Once: O(1). Two consecutive Fibonacci values.
3	`steps = 0`	Once: O(1).
4	`for _ in range(n):`	n body visits; range itself needs O(1) space.
5	`a, b = b, a + b`	n times × O(1) = Θ(n); one Fibonacci addition per visit.
6	`steps += 1`	n times × O(1) = Θ(n).
7	`return a, steps`	Once: O(1); returns two values, not a growing list.

Calculation

T(n) = c0 + c1·n + c2·n = a·n + b = Θ(n). There is one loop, so its body counts are added, not multiplied by each other. Space: a, b, steps and the loop state occupy a fixed number of words: Θ(1) auxiliary space in this model.

Example: n = 7

Loop visits = 7; additions a+b = 7; counter increments = 7; returned pair = (13, 7). “7 steps” measures loop-body executions only, not all individual operations.

A1 | Naive recursive Fibonacci

Line	Code statement	Frequency and cost / explanation
1	`def recursive(n):`	Let C(n) be the number of calls, including base cases.
2	`if n <= 1:`	Once in every call: total C(n) tests, each O(1).
3	`return n, 1`	Once per leaf call; base-case work is Θ(1).
4	`x, calls_x = recursive(n - 1)`	One recursive child per non-base call: cost T(n-1).
5	`y, calls_y = recursive(n - 2)`	Second child per non-base call: cost T(n-2).
6	`return x + y, 1 + calls_x + calls_y`	O(1) local work per non-base call; combines values and call counts.

Recurrence and solution

T(n) = T(n-1) + T(n-2) + Θ(1), with T(0), T(1) = Θ(1). The characteristic growth factor is φ = (1+√5)/2 ≈ 1.618. Thus T(n) = Θ(φ^n). O(2^n) is a valid, looser upper bound.

C(0)=C(1)=1; C(n)=1+C(n-1)+C(n-2)=2F(n+1)-1. For n=7: C(7)=2F(8)-1=2×21-1=41 calls. There are 20 non-base calls and 21 leaves. Calls grow exponentially because subproblems repeat.

Space calculation

The deepest chain decreases n by one each time. At most Θ(n) frames are live together, each with a constant number of local values: Θ(n) stack space. The two branches run sequentially, so exponential total calls do not imply exponential live stack space.

A1 | Driver costs and comparison

Input and validation

n=int(input(...)): one integer read and conversion. The two condition checks and raise SystemExit on invalid input are constant under the unit-cost model; text reading depends on the number of input digits.

Series construction, line by line

series=[]: O(1). a,b=0,1: O(1). for _ in range(n+1): n+1 visits. series.append(a): amortized O(1) per visit, Θ(n+1) total. a,b=b,a+b: Θ(n+1) arithmetic operations. print("Series:",series): visits n+1 entries, plus numeric formatting.

Function calls and final print lines

value,steps=iterative(n): Θ(n). Printing the pair is bounded work apart from integer formatting. value,calls=recursive(n): Θ(φ^n). Printing those results does not change the dominant operation-count growth.

Whole program

T(n)=Θ(n)+Θ(n)+Θ(φ^n)=Θ(φ^n). The stored series and recursive call stack each use Θ(n) words; their sum remains Θ(n). Iterative function alone uses Θ(1) auxiliary words.

Important integer detail

F(n) has Θ(n) bits. If bit costs are counted, repeated large-integer additions are not O(1). State “standard unit-cost arithmetic model” when giving the textbook complexity. The printed series also contains $\Theta(n^2)$ total numeric bits.

A2 | Job sequencing with deadlines

Line	Code statement	Frequency and cost / explanation
1	`def schedule_jobs(jobs):`	n jobs; D = min(n, maximum deadline), assuming nonnegative integer deadlines. Empty input gives D=0.
2	`jobs = sorted(jobs, key=lambda job: job[2], reverse=True)`	n key evaluations: Θ(n). Sorting: O(n log n) worst case; new list: Θ(n) space.
3	`size = min(len(jobs), max((job[1] for job in jobs), default=0))`	Generator visits n jobs: Θ(n) time, O(1) generator state. len and min are O(1). size=D.
4	`slots = [None] * size`	Θ(D) initialization time and Θ(D) storage.
5	`total_profit = 0`	Once: O(1).
6	`for name, deadline, profit in jobs:`	n visits, each tuple unpacking O(1).
7	`if profit <= 0:`	n constant-time tests.
8	`continue`	O(1) when reached; skips the slot search for this job.
9	`for slot in range(min(deadline, size) - 1, -1, -1):`	Job i visits $q_i \le D$ slots; all searches cost $\Sigma q_i \le nD$.
10	`if slots[slot] is None:`	One O(1) test per searched slot.
11	`slots[slot] = name`	O(1) per successful job; at most min(n,D) successes.
12	`total_profit += profit`	O(1) per successful job.
13	`break`	O(1); ends this search after scheduling.
14	`return slots, total_profit`	O(1); returns a reference to slots, not a copy.

A2 | Complexity calculation

Total time

$T(n,D)=O(n \log n)+\Theta(n)+\Theta(D)+\Theta(n)+O(\Sigma q_i) = O(n \log n+nD)$. As $D \leq n$, worst-case time is $O(n^2)$. A break may shorten some searches; it does not remove the worst-case bound.

Why a quadratic search can occur

Let every deadline be n and every profit be positive. The first job checks 1 slot, the second 2, and so on. Total slot tests = $1+2+...+n = n(n+1)/2 = \Theta(n^2)$.

Calculation on the actual sample

Profit order: A(100), C(27), D(25), B(19), E(15). D=3. A tests slot index 1: 1 check. C tests 1 then 0: 2 checks. D tests 0: 1 check. B tests 0: 1 check. E tests 2: 1 check. Total=6 slot checks. Slots=[C,A,E], profit=142. Sorting and setup are additional work.

Space

sorted creates $\Theta(n)$ references and may use $O(n)$ sorting workspace. slots uses $\Theta(D)$. Other variables are $O(1)$. Auxiliary storage including the returned slots is $O(n+D)=O(n)$. Excluding output, the sorted copy still requires $O(n)$.

Driver statements

jobs=[...]: $\Theta(n)$ to construct generalized input, $\Theta(n)$ input space. slots,profit=schedule_jobs(jobs): cost above. print("Time slots:",slots): $\Theta(D)$ entries plus formatting. print("Maximum profit:",profit): one value. Literal five-job demo is fixed-sized.

Best-case caution

Python sorting can be linear on already ordered data. If the slot searches are skipped or very short, the function can be $\Theta(n)$. “$O(n \log n+nD)$” is a worst-case upper bound, not an exact count for every input.

A3 | Fractional knapsack

Line	Code statement	Frequency and cost / explanation
1	`def fractional_knapsack(items, capacity):`	n items, positive weights and nonnegative capacity. Let $k \le n$ be the number selected.
2	`items = sorted(items, key=lambda item: item[2] / item[1], reverse=True)`	n ratio computations: $\Theta(n)$. Sort: $O(n \log n)$ worst-case time; $O(n)$ new-list space.
3	`total = 0.0`	$O(1)$, once.
4	`selected = []`	$O(1)$, initially empty.
5	`for name, weight, value in items:`	At most n body visits; $O(1)$ unpacking per visit.
6	`if capacity == 0:`	$O(1)$ per visit.
7	`break`	$O(1)$, if capacity is exhausted.
8	`taken_weight = min(weight, capacity)`	k executions, each $O(1)$.
9	`fraction = taken_weight / weight`	k executions, each $O(1)$.
10	`total += fraction * value`	k executions, each $O(1)$.
11	`selected.append((name, fraction))`	k amortized $O(1)$ appends: $\Theta(k)$ total and $\Theta(k)$ storage.
12	`capacity -= taken_weight`	k constant-time updates.
13	`return total, selected`	$O(1)$; returns the existing list by reference.

Calculation and example

$T(n)=O(n \log n)+O(n)=O(n \log n)$ worst case. A zero capacity still does not avoid sorting: the sort happens before the loop. Space=$O(n)$ sorted references + $O(k)$ selections = $O(n)$.

For A=(10,60), B=(20,100), C=(30,120), ratios are 6, 5, 4. Capacity 50: take 10, then 20, then 20. Three ratio evaluations and three selection updates; value=$60+100+(20/30)\times120=240$. The scan is linear; sorting dominates worst-case growth.

Driver

items=[...]: $\Theta(n)$ input creation. Function call: cost above. for name,fraction in selected and its print: k visits, $\Theta(k)$ output entries. Final value print: one value. Formatting costs depend on the number of characters produced.

A4 | 0/1 knapsack with a 2D table

Line	Code statement	Frequency and cost / explanation
1	`def knapsack(weights, values, capacity):`	n items; W=capacity, a nonnegative integer.
2	`n = len(weights)`	O(1).
3	`dp = [[0] * (capacity + 1) for _ in range(n + 1)]`	n+1 rows × W+1 cells: Θ((n+1)(W+1)) time and space.
4	`for i in range(1, n + 1):`	n body visits.
5	`for w in range(capacity + 1):`	W+1 visits per item: n(W+1) total.
6	`dp[i][w] = dp[i - 1][w]`	n(W+1) constant-time assignments.
7	`if weights[i - 1] <= w:`	n(W+1) O(1) tests.
8	`include = values[i - 1] + dp[i - 1][w - weights[i - 1]]`	At most n(W+1) executions, each O(1).
9	`dp[i][w] = max(dp[i][w], include)`	Same conditional count; compares two values: O(1).
10	`selected = []`	Once: O(1).
11	`w = capacity`	Once: O(1).
12	`for i in range(n, 0, -1):`	n traceback visits.
13	`if dp[i][w] != dp[i - 1][w]:`	n O(1) comparisons.
14	`selected.append(i)`	At most n amortized O(1) appends.
15	`w -= weights[i - 1]`	At most n O(1) updates.
16	`return dp[n][capacity], list(reversed(selected))`	O(k) time and new output space for k selected items; k≤n.

A4 | Complexity calculation

Count the table cells

Allocation=(n+1)(W+1). DP updates=n(W+1). Let I=Σ max(0,W-weight_i+1) for nonnegative integer weights; the include branch executes I≤n(W+1) times. Traceback checks=n.

Add the terms

$T(n,W)=\Theta((n+1)(W+1))+\Theta(n(W+1))+O(n)=\Theta((n+1)(W+1))$. For $n,W\geq 1$ this is $\Theta(nW)$. Retaining +1 handles W=0 correctly: this code still initializes and scans Θ(n) cells.

Actual sample calculation

n=3, W=50. Allocated cells=4×51=204. Updated cells=3×51=153. Include branch counts: weight 10 gives 41, weight 20 gives 31, weight 30 gives 21; total=93. Traceback checks=3. Selected item numbers=[2,3]; value=220.

Space

dp stores (n+1)(W+1) values. selected and the returned reversed copy each hold at most n entries. Loop variables use O(1). Peak space=Θ((n+1)(W+1)); normally written Θ(nW). Returning does not copy the DP table.

Why pseudo-polynomial?

W is a numeric capacity. Writing W in binary requires about $\log_2 W$ bits. A loop of W iterations is not polynomial in $\log_2 W$. Therefore this is pseudo-polynomial, even though it is polynomial in n and the numeric value W.

Driver costs

weights=[...], values=[...]: Θ(n) input construction. value,selected=knapsack(...): cost above. Printing value is one value; printing selected visits k≤n entries. These do not dominate table construction and processing.

A5 | Binomial coefficient using DP

Line	Code statement	Frequency and cost / explanation
1	`def binomial(n, r):`	Assume n≥0. Valid r satisfies 0≤r≤n.
2	`if r < 0 or r > n:`	One O(1) validity check.
3	`return 0`	O(1) early return for invalid r; no table allocated.
4	`dp = [[0] * (r + 1) for _ in range(n + 1)]`	(n+1)(r+1) zero entries: Θ((n+1)(r+1)) time and space.
5	`for i in range(n + 1):`	n+1 row visits.
6	`for j in range(min(i, r) + 1):`	Row i visits min(i,r)+1 cells; total S=Σ[min(i,r)+1].
7	`if j == 0 or j == i:`	S constant-time boundary tests.
8	`dp[i][j] = 1`	O(1) each time a valid boundary is visited.
9	`else:`	Selects the recurrence branch; no extra loop.
10	`dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]`	O(1) arithmetic and indexing per interior cell in the unit-cost model.
11	`return dp[n][r]`	O(1); returns one value, not the table.

Summation, including the triangular part

For 0≤r≤n: S = Σ(i=0..r)(i+1) + Σ(i=r+1..n)(r+1) = (r+1)(r+2)/2 + (n-r)(r+1) = (r+1)(n+1-r/2). Thus S=Θ((n+1)(r+1)). Allocation already has this order. Time and space are both Θ((n+1)(r+1)); for n,r≥1, Θ(nr).

Example and edge cases

n=5,r=2: allocation=6×3=18 cells. Visits per row=1,2,3,3,3,3; S=15. Boundary writes=8 and addition cells=7; C(5,2)=10. When r=0, this implementation still allocates and visits n+1 cells, so it takes Θ(n), not zero time.

Driver

Read n and r: input-length cost; n<0 check and error exit: O(1). The final print calls binomial once: table cost above, then formats one result. Big-integer additions can exceed O(1) when bit complexity is counted.

A6 | Queens: board and safety check

Line	Code statement	Frequency and cost / explanation
1	`def solve_queens(n, fixed_row, fixed_col):`	Analyze generalized N=n queens, with one fixed queen.
2	`board = [-1] * n`	Θ(N) initialization time and Θ(N) storage.
3	`board[fixed_row] = fixed_col`	O(1).
4	`def safe(row, col):`	One safety check for one trial placement.
5	`for r, c in enumerate(board):`	At most N body visits; enumerate does not copy board.
6	`if c == -1 or r == row:`	O(1) per visited row.
7	`continue`	O(1); skips empty rows and the same row.
8	`if c == col or abs(r - row) == abs(c - col):`	O(1) comparison/arithmetic per occupied row.
9	`return False`	O(1) early exit on first conflict.
10	`return True`	O(1), after scanning the complete board.

Cost of safe

A conflict can be found immediately: best case Θ(1). If no conflict is found, the loop visits all N board entries: worst case Θ(N). Empty rows are still visited and tested. No set or diagonal lookup structure is used in this version, so safety is not O(1).

A call to safe uses constant extra local storage, Θ(1). It reads the existing Θ(N)-element board. The board is not copied for each attempt.

A6 | Queens: recursive search

Line	Code statement	Frequency and cost / explanation
1	`def place(row):`	One search state handles one row; the recursion proceeds in row order.
2	`if row == n:`	O(1) per call; detects a complete solution.
3	`return True`	O(1); first solution terminates further searching.
4	`if row == fixed_row:`	O(1) test per call.
5	`return place(row + 1)`	One recursive continuation; no column trials in the fixed row.
6	`for col in range(n):`	At most N trial columns per non-fixed search state.
7	`if safe(row, col):`	At most N calls per state × O(N) each = $O(N^2)$ local work.
8	`board[row] = col`	O(1) for each safe choice.
9	`if place(row + 1):`	Recursive cost for the next row; explored only after a safe choice.
10	`return True`	O(1); propagates success.
11	`board[row] = -1`	O(1); undoes an unsuccessful choice.
12	`return False`	O(1), after exhausting that state.
13	`return board if place(0) else None`	Calls the search once; returning board is O(1), with no copy.

Why the running time is not just N^2

$O(N^2)$ is the local work at one state. Backtracking creates many states. Because occupied columns cannot be reused, the number of valid partial column assignments is bounded by permutations. Summing depths gives O(N!) states as a conservative bound, even before diagonal pruning. Multiply state count by $O(N^2)$ local work: $O(N^2 \cdot N!)$ time.

A6 | Bound, example and memory

Interpret the bound correctly

$O(N^2 \cdot N!)$ is a safe upper bound for this scan-based implementation, not a claim that every board explores that many states. One fixed queen and diagonal rejection reduce exploration; returning the first solution can stop much earlier. The common $O(N!)$ description abstracts away trial and safety-check overhead.

Simple numerical bound illustration

For $N=8$: $8!=40{,}320$. Multiplying the upper-bound factors gives $8^2 \times 8! = 2{,}580{,}480$. This illustrates the formula only; it is not an exact count of checks or instructions. For one safe call, at most 8 board entries are scanned; one search state tries at most 8 columns.

Space calculation

The board has N entries. Search depth is at most N+1 calls. Each call stores O(1) local state; a safe call adds O(1) working storage and does not keep a second recursive tree alive. Total peak auxiliary space, including returned board, is $\Theta(N)$.

Main input and validation

Read row and col, subtract 1: O(1) arithmetic plus input parsing. The four bounds checks and invalid-input exit are O(1). solve_queens(8,row,col) uses the search above; to discuss growth, replace literal 8 by N.

Printing the solution

if answer is None and its print: O(1) aside from text output. Otherwise for c in answer has N visits. Each join generates N board symbols, so board display is $\Theta(N^2)$ time and O(N) temporary row-text storage. [c+1 for c in answer] takes $\Theta(N)$ time and space. Printing the board can dominate an unusually short successful search, but not the stated worst-case search bound.

Exam wording

“This is backtracking. Each state may try N columns, each safety test scans N rows, and there are factorially many possible partial placements. A conservative bound is $O(N^2\ N!)$, with O(N) space.”

A7a | Matrix multiplication: core work

Line	Code statement	Frequency and cost / explanation
1	`def multiply(a, b, threaded=False):`	A has r rows and k columns; B has k rows and c columns. Assume positive compatible dimensions.
2	`rows, inner, cols = len(a), len(b), len(b[0])`	O(1); len of a Python list does not scan its entries.
3	`if any(len(row) != inner for row in a):`	At most r O(1) length checks: O(r); generator storage O(1).
4	`raise ValueError(...)`	O(1) on an invalid dimension; exits before multiplication.
5	`result = [[0] * cols for _ in range(rows)]`	r rows × c entries: Θ(rc) initialization time and output space.
6	`def calculate_row(i):`	Worker computes a single output row.
7	`for j in range(cols):`	c visits per worker; rc across all r rows.
8	`for k in range(inner):`	k visits per cell; rck across the complete multiplication.
9	`result[i][j] += a[i][k] * b[k][j]`	rck O(1) updates; each has one multiplication and one addition.

Summation

Total arithmetic updates = $\Sigma(i=0..r-1)\ \Sigma(j=0..c-1)\ \Sigma(t=0..k-1)\ 1 = rck$. Core work = $\Theta(rck)$; including validation and output initialization gives $\Theta(r+rc+rck)=\Theta(rck)$ for positive dimensions.

Numerical example

For two 2×2 matrices, r=c=k=2: output initialization writes 4 zeros, then 2×2×2=8 multiply-add updates. Result=[[19,22],[43,50]]. For n×n matrices, updates=n^3, so doubling n multiplies arithmetic work by 8.

A7a | Serial path and thread path

Line	Code statement	Frequency and cost / explanation
1	`if threaded:`	O(1) selection.
2	`threads = [Thread(target=calculate_row, args=(i,)) for i in range(rows)]`	r thread objects: O(r) setup under a fixed-cost thread model, O(r) management space.
3	`for thread in threads:`	r iterations.
4	`thread.start()`	r starts. Startup has runtime overhead; worker arithmetic is counted separately.
5	`for thread in threads:`	r join operations.
6	`thread.join()`	Waits for completion; do not label elapsed waiting O(1). It does not duplicate worker work.
7	`else:`	Serial branch.
8	`for i in range(rows):`	r visits.
9	`calculate_row(i)`	Θ(ck) work per row; r calls give Θ(rck).
10	`return result`	O(1); existing matrix reference returned.

Time and space

Serial time=Θ(rck). Threaded total arithmetic work=Θ(rck), plus thread setup and scheduling overhead. With ideal independent execution and p processors, a work-based lower bound is Ω(rck/p), and one-row workers also impose Ω(ck) critical-path work. This is not a guaranteed Python speedup.

Output storage=Θ(rc). Serial auxiliary working storage excluding output is Θ(1). Threaded management and per-worker local state use O(r), excluding OS-dependent stack reservations; including output gives O(rc+r). The matrix inputs use Θ(rk+kc), counted separately.

A7a | Benchmark and driver analysis

benchmark(a,b,threaded), line by line

times=[]: O(1). for _ in range(3): exactly three repetitions. start=perf_counter(): constant-sized timer call. multiply(...): Θ(rck) arithmetic work each. times.append(perf_counter()-start): O(1) amortized, three values. return median(times): median of three numbers, O(1). Thus 3×Θ(rck)+O(1)=Θ(rck), with the same matrix-space order.

Imports and fixed demo

Thread, perf_counter, median and Random imports are setup independent of matrix dimensions. Constructing the two literal 2×2 matrices is bounded. Each result print calls multiply once and traverses four output entries.

Generalized size-n benchmark loop

rng=Random(42): fixed setup. for n in [20,40,60]: three fixed size cases. Each a/b nested comprehension generates n^2 entries: $\Theta(n^2)$ time and space per matrix. assert multiply(a,b)==multiply(a,b,True): two $\Theta(n^3)$ multiplications plus up to $\Theta(n^2)$ element comparisons.

Remaining benchmark statements

serial=benchmark(...,False) and parallel=benchmark(...,True) each perform three multiplications: $\Theta(n^3)$ work each. The formatted print computes serial/parallel and formats a few numbers. Final explanatory print is fixed text. Per size, total arithmetic work remains $\Theta(n^3)$; retaining input and output matrices needs $\Theta(n^2)$ space.

Threading interpretation

Two points must be separated: total work and elapsed time. The code still performs n^3 arithmetic updates. Thread startup, scheduling, and the Python runtime influence timing. The ratio serial_time/thread_time is a measured speedup; it does not change the work-complexity formula.

A7b | Merge two sorted lists

Line	Code statement	Frequency and cost / explanation
1	`def merge(left, right):`	Let p=len(left), q=len(right), s=p+q.
2	`result = []`	O(1).
3	`i = j = 0`	O(1).
4	`while i < len(left) and j < len(right):`	h successful body visits, h≤s-1 when both inputs are nonempty. O(1) per test.
5	`if left[i] <= right[j]:`	h comparisons, O(1) each.
6	`result.append(left[i])`	One branch appends on each visit; amortized O(1).
7	`i += 1`	O(1) when the left branch is taken.
8	`else:`	Constant-time branch control.
9	`result.append(right[j])`	Amortized O(1) when the right branch is taken.
10	`j += 1`	O(1) when the right branch is taken.
11	`return result + left[i:] + right[j:]`	Slices copy remaining entries; + creates lists and copies references. Whole expression is O(s) time and O(s) peak extra storage.

Why the final line matters

Even if the comparison loop ends early, leftover slicing and concatenation still copy the remaining elements. Total time=Θ(p+q); output size=p+q. Temporary lists add only a constant multiple of p+q, so peak extra storage is O(p+q), not quadratic.

Example

left=[3,27,38], right=[9,10,43,82]. Comparison-loop picks: 3,9,10,27,38; h=5. Then [43,82] remains. Five loop appends plus two leftover entries give seven output elements; concatenation also copies references already in result.

A7b | Recursive merge sort

Line	Code statement	Frequency and cost / explanation
1	`def merge_sort(a):`	n=len(a); analyze one call.
2	`if len(a) <= 1:`	O(1) at every recursive node.
3	`return a[:]`	Base case copies zero or one entry: O(1).
4	`mid = len(a) // 2`	O(1) at every non-base node.
5	`return merge(merge_sort(a[:mid]), merge_sort(a[mid:]))`	Two slices together copy n references: Θ(n). Two recursive sorts plus a Θ(n) merge. Local nonrecursive work is Θ(n).

Solve the recurrence

$T(n)=T(\lfloor n/2 \rfloor)+T(\lceil n/2 \rceil)+\Theta(n)$. For n a power of two: $T(n)=2T(n/2)+c\cdot n$. There are $\log_2 n$ split levels, each processing Θ(n) total elements. Add the leaf work Θ(n): $T(n)=\Theta(n \log n)$, for $n \geq 2$.

For n=8: non-base levels process 8, 2×4=8, and 4×2=8 elements. Total level-size sum=24=$8\log_2 8$. There are 8 leaves. This is a growth calculation, not an exact count of all assignments and copies.

Best, worst and space

Best and worst time are Θ(n log n). The code always splits and copies slices, even when input is already sorted. Live slice sizes down one branch form $n+n/2+n/4+\ldots<2n$; pending results and merge temporaries add another O(n). Peak auxiliary storage is O(n), including the returned list; stack depth is O(log n). Total allocations over the whole run can be Θ(n log n), but peak live storage is O(n).

A7b | Two-worker threaded merge sort

Line	Code statement	Frequency and cost / explanation
1	`def threaded_sort(a):`	n input elements; only two top-level workers.
2	`mid = len(a) // 2`	O(1).
3	`halves = [None, None]`	O(1); two references, not n elements.
4	`def sort_half(index, part):`	Worker helper.
5	`halves[index] = merge_sort(part)`	Sort each half: Θ((n/2) log(n/2)) work; assignment itself O(1).
6	`t1 = Thread(target=sort_half, args=(0, a[:mid]))`	Slice Θ(n) for a half; thread creation has fixed management overhead.
7	`t2 = Thread(target=sort_half, args=(1, a[mid:]))`	Other half slice Θ(n); together n copied entries.
8	`t1.start()`	Starts worker 1; worker sorting cost is counted above.
9	`t2.start()`	Starts worker 2.
10	`t1.join()`	Waits for worker 1; elapsed waiting depends on remaining work.
11	`t2.join()`	Waits for worker 2.
12	`return merge(halves[0], halves[1])`	Final Θ(n) merge; creates the final output.

Calculation

Total work=$2T(n/2)+\Theta(n)=\Theta(n \log n)$. Even with ideal simultaneous workers, elapsed compute is $T(n/2)+\Theta(n)=\Theta(n \log n)$: a fixed number of workers changes a constant, not the asymptotic class. Peak auxiliary list storage remains O(n), with O(log n) recursion depth per worker and two thread objects.

A7b | Benchmark and driver costs

benchmark(function,data), first lines

times=[]: O(1). for _ in range(3): three runs. start=perf_counter(): O(1). answer=function(data): Θ(n log n) each for either sort. times.append(perf_counter()-start): O(1) amortized; three retained times.

Verification and return

assert answer==sorted(data): builds a sorted reference copy in O(n log n) worst-case time, O(n) storage; list equality checks up to n elements. This assertion is after timing collection and is not part of the individual stored sort duration. return median(times): O(1), as the list contains exactly three values. Benchmark overall remains Θ(n log n).

Main block: setup and cases

The first print sorts and prints seven fixed entries. Random(42): fixed setup. for n in [1000,4000,8000]: three size cases. cases creates sorted and reverse lists via list(range(...)) and a random list via n random draws: Θ(n) time and space per size.

Main block: measurements

for case,data in cases.items(): three cases, a constant. serial=benchmark(merge_sort,data) and threads=benchmark(threaded_sort,data): Θ(n log n) work each. Printing case labels and two times is fixed-field formatting. Final explanatory prints are fixed text.

Overall and input order

Per variable size n, constant repetitions and cases preserve Θ(n log n) total work and O(n) peak auxiliary storage. Sorted and reverse inputs may change comparison counts; they do not change this implementation’s Θ(n log n) class. Timing differences do not prove a different asymptotic complexity.

A7c | Naive string matching

Line	Code statement	Frequency and cost / explanation
1	`def naive(text, pattern):`	n=text length, m=pattern length. First analyze 1≤m≤n. Let L=n-m+1.
2	`n, m = len(text), len(pattern)`	O(1); Python string lengths are stored.
3	`matches = []`	O(1).
4	`for i in range(n - m + 1):`	L alignments.
5	`for j in range(m):`	At alignment i, a_i comparisons, 1≤a_i≤m.
6	`if text[i + j] != pattern[j]:`	O(1) per comparison; total Σa_i.
7	`break`	O(1) at a mismatch; ends only the inner loop.
8	`else:`	Python for-else executes when the inner loop has no break.
9	`matches.append(i)`	Once per match: z≤L amortized O(1) appends.
10	`return matches`	O(1); returns existing list.

Calculation

T(n,m)=Θ(L+Σa_i). Best case: first character differs at every alignment, so Σa_i=L and time=Θ(L). Worst case: m comparisons at each alignment, so time=Θ(Lm)=Θ((n-m+1)m). O(nm) is the customary looser upper bound.

Example worst case: text="AAAAAA" (n=6), pattern="AAA" (m=3). L=4, comparisons=4×3=12, matches=[0,1,2,3]. Example early mismatch: pattern="BAA" gives 4 comparisons. If m=n, there is one alignment, so the tight worst bound is Θ(n), although O(nm) still holds.

Space and edge cases

O(1) working space excluding matches; Θ(z) output storage. No substring slice is made. Empty pattern m=0: the inner loop is empty, so every boundary is appended: Θ(n+1) time and output. If m>n, the outer range is empty: Θ(1) function time and empty output.

A7c | Rabin-Karp: setup and initial hashes

Line	Code statement	Frequency and cost / explanation
1	`def rabin_karp(text, pattern):`	n text characters; m pattern characters.
2	`n, m = len(text), len(pattern)`	O(1).
3	`if m == 0:`	O(1) check.
4	`return list(range(n + 1))`	Empty pattern: Θ(n+1) time and output storage.
5	`if m > n:`	O(1) check.
6	`return []`	O(1) for pattern longer than text.
7	`base, prime = 256, 101`	O(1); fixed-size hash parameters.
8	`high = pow(base, m - 1, prime)`	O(log m) modular multiplications via binary exponentiation; O(1) modular arithmetic per multiplication here.
9	`pattern_hash = window_hash = 0`	O(1).
10	`for j in range(m):`	Exactly m iterations.
11	`pattern_hash = (base * pattern_hash + ord(pattern[j])) % prime`	m O(1) fixed-modulus updates.
12	`window_hash = (base * window_hash + ord(text[j])) % prime`	m O(1) fixed-modulus updates.

Setup calculation

Setup work=Θ(m)+O(log m)=Θ(m) for m≥1, treating the m=1 power as constant. Two hash-update statements run m times each: 2m fixed-size updates. Hash values remain bounded by prime=101; this is why hash arithmetic can be treated as constant here.

The function stores two hashes, a power, and a few indices: O(1) persistent working variables. Substring verification in the next part introduces extra temporary space.

A7c | Rabin-Karp: scan and verification

Line	Code statement	Frequency and cost / explanation
1	`matches = []`	O(1); let z be returned match count.
2	`for i in range(n - m + 1):`	L=n-m+1 window visits.
3	`if pattern_hash == window_hash and text[i:i + m] == pattern:`	Hash test O(1) every window. Short-circuiting creates a slice only for equal hashes. Each such slice copies m characters: Θ(m), plus up to O(m) equality work.
4	`matches.append(i)`	z amortized O(1) appends; verified matches only.
5	`if i < n - m:`	L O(1) tests; excludes update after the last window.
6	`window_hash = (base * (window_hash - ord(text[i]) * high) + ord(text[i + m])) % prime`	L-1 fixed-size rolling updates: Θ(L-1). It does not recompute all m characters.
7	`return matches`	O(1), returns existing list.

Input-sensitive formula

Let h be the number of equal-hash windows, including genuine matches and collisions. $T(n,m,h)=\Theta(m+L+h\cdot m)$, because the slice alone costs Θ(m) on each verification. With few enough candidates that $h\cdot m=O(n+m)$, time is Θ(n+m). Worst case h=L: Θ(m+Lm), commonly bounded by O(nm).

The code uses fixed prime 101 and no randomized hash choice. Linear expected time therefore requires assumptions on the inputs/candidate frequency; it is not an unconditional guarantee. Many genuine matches can also make verification expensive, even without collisions.

A7c | Worked count, memory and driver

Sample from the source

text="AABAACAADAABAABA": n=16. pattern="AABA": m=4. L=16-4+1=13 windows. Matches=[0,9,12]. Initial hash statements perform 2m=8 updates; rolling updates=L-1=12. With base=256, prime=101, the pattern hash is 79; equal-hash windows in this sample are exactly 0,9,12, so h=3.

Verification work

The sample makes three length-4 substring copies: 3×4=12 copied characters, plus equality checks. The formula's size terms are m+L+h·m=4+13+12=29 units of growth terms; this is not a complete primitive-operation count. On text="AAAAAA", pattern="AAA", all four windows verify and verification copies 12 characters.

Space calculation

Persistent hash state: O(1). Python text[i:i+m] creates a temporary substring of size O(m). Output list stores z indices. Therefore peak extra storage including output is O(m+z), and auxiliary working space excluding output is O(m). This differs from a character-by-character verification that could avoid the slice.

Driver, line by line

perf_counter import: setup. text=input(...), pattern=input(...): Θ(n+m) reading and storage. for method in [naive,rabin_karp]: two methods, constant. start=perf_counter(): O(1). for _ in range(100): fixed 100 repetitions. positions=method(...): the corresponding matching cost each time; output is replaced rather than storing 100 results.

Average time and printing

seconds=(perf_counter()-start)/100: O(1). print(method.__name__,positions,...): z output indices plus formatting. Final print is fixed text. Repeating a function 100 times multiplies runtime by 100 but leaves its asymptotic class unchanged. Peak storage may briefly include old and new match lists, still O(z+m) for Rabin-Karp.

What to say in viva

"Naive compares characters at each alignment. Rabin-Karp updates hashes in constant work per shift but verifies equal hashes. With few candidates it is linear; worst-case verification gives O(nm). This Python code uses O(m) temporary slice storage."

Revision | Final complexity map

Bounds refer to the functions under the standard unit-cost model. For concise formulas below assume positive nontrivial input sizes. See individual sections for zero inputs and tighter input-sensitive bounds.

Program	Time	Extra space (including output)
Iterative Fibonacci	$\Theta(n)$	$\Theta(1)$ words
Recursive Fibonacci	$\Theta(\varphi^n)$; $O(2^n)$ upper bound	$\Theta(n)$ stack words
Job sequencing	$O(n \log n+nD)$; worst $\Theta(n^2)$	$O(n+D)$, $D \leq n$
Fractional knapsack	$O(n \log n)$ worst case	$O(n)$
0/1 knapsack DP	$\Theta((n+1)(W+1))$	$\Theta((n+1)(W+1))$
Binomial DP, valid r	$\Theta((n+1)(r+1))$	$\Theta((n+1)(r+1))$
Queens, scan-based safety	$O(N^2 \cdot N!)$ conservative upper bound	$\Theta(N)$
Matrix, serial/thread work	$\Theta(rkc)$; square $\Theta(n^3)$	Serial $\Theta(rc)$; threaded $O(rc+r)$
Merge sort, either version	$\Theta(n \log n)$, best and worst	$O(n)$ lists; $O(\log n)$ depth
Naive matching, $1 \leq m \leq n$	Worst $\Theta((n-m+1)m)$	$O(1+z)$
Rabin-Karp, $1 \leq m \leq n$	$\Theta(m+L+hm)$; worst $O(nm)$	$O(m+z)$

Five rules for calculating complexity

1. Count visits before simplifying: $n(W+1)$, not “two loops means n^2”.
2. Charge built-ins for their real work: sorted, slices, concatenation and list construction are not $O(1)$.
3. Add sequential work; for recursive calls, solve the recurrence.
4. Count maximum live memory, not every allocation made during the entire run.
5. State the input variables, arithmetic model, and whether output is included.

Source and analysis scope

Source: the attached LP3_Complete_Practical.pdf, printed pages 7-23 (Group A). Code statements are reproduced for complexity analysis; comments and long message strings are abbreviated where they do not affect cost. Calculations are derived from those implementations. No ML or blockchain material is included.